Something Big

Attempter Guidelines

Change Log

Date	Changes	Author
Mar 20, 2026	Added instructions for Docker extensions	Jesus
Mar 14, 2026	Common errors section	Matin
Mar 11, 2026	New instruction document release.	Antonio

Change Log 1
Core Objectives 2
Important Definitions 2
1. Implementation-Agnostic ✔️ 2
2. Implementation-Specific ❌ [MUST AVOID] 3
Comparison Table 3
3. Test Coverage 3
Task Types 4
1. Full Tool Implementation 4
Important Commands for this Project 4
Workflow Overview 5
1. Full Tool Implementation 5
3. Full Tool Implementation Workflow 5
Rubric Generation 6
🚨YOU DON`T NEED TO CREATE RUBRICS FOR THINGS THAT ARE ALREADY BEING

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

COVERED BY UNIT TESTS!🚨 - you will not be penalized if you do..... 6
Rubric Scoring..... 7
Rubric Weight..... 7
Rubric Dimension..... 7
Examples..... 8
1. Instruction Following..... 8
2. Code Correctness..... 8
Rubric basics..... 9
Complexity Level..... 10
The Prompt..... 10
1. The "Architectural" Mindset..... 11
2. The Structure of a Complex Prompt..... 11
3. Mandatory Component: Expected Interface..... 11
Required Fields..... 11
Language-Specific Fields (Include as applicable)..... 12
4. Complexity Checklist..... 12
5. Summary Checklist for a High-Quality Prompt..... 12
Audit Script..... 12
Extra Validation Step [FOR REVIEWERS ONLY]..... 16

Core Objectives

This project involves assessing, testing, and refining code based on specific task requirements. Since scenarios vary, ranging from comparing repository commits to creating unit tests from scratch. You should follow the unique guidelines provided for each task.

Important Definitions

Your prompt should describe only the **minimal external interaction surface** — what the solution must do, not how it must be built.

1. Implementation-Agnostic ✔️

Describes WHAT the solution must do, not HOW the model must build it. You still document the relevant external interfaces, such as classes and functions, but only at the level needed to define their role, inputs, outputs, or expected behavior. You do not prescribe internal logic, implementation steps, helper functions, private methods, getters, or setters. The focus is only on the minimal external interaction surface that the evaluation uses in practice.

Example of a **Implementation-Agnostic** Prompt:

```
None
Implement a `WordCounter` class that accepts a string on initialization.
It must expose a `count(word)` method that returns the number of times
the given word appears in the string (case-insensitive).
```

This only specifies the external contract: the constructor input and the method's signature + behavior. The model is free to use .split(), regex, a dictionary, or any other approach internally.

2. Implementation-Specific ❌ [MUST AVOID]

Describes not only WHAT the solution must do, but also HOW it should be implemented. Instead of staying at a high-level contract, it constrains the internal design or logic of the function or class, such as the exact method structure, required internal steps, helper functions, control flow, data handling, or other design choices.

Example of an **Implementation-Specific** Prompt:

```
None
Implement a `WordCounter` class that accepts a string on initialization.
Store the string as `self._text`. Implement a private `_normalize()` method
that lowercases the string, then split it by whitespace into `self._words`.
The `count(word)` method should iterate over `self._words` and use a
counter variable to tally matches before returning it.
```

This over-specifies the internals — field names, a required private method, and the exact counting logic. Any solution that doesn't follow these steps would fail evaluation, even if it produces correct results.

Comparison Table

	Agnostic ✅	Specific ❌
Constructor	accepts a string	store as self._text
Methods	count(word) → int	must call self._normalize() first
Logic	not specified	use a counter variable in a loop
Data structures	not specified	split into self._words list

3. Test Coverage

Test coverage means your tests collectively exercise every line of the implementation through its public interface — if a line of code runs during a test, it's covered, regardless of which internal path triggered it. This means you never need to write dedicated tests for helper functions or private methods, because they get exercised automatically when you test the main behavior. For example, if `count(word)` internally calls `_normalize()` and `_split()`, a test like `assert counter.count("Hello") == 2` already covers all three — writing a separate test for `_normalize()` would be redundant and would also violate **implementation-agnosticism** by reaching into internals.

Test Coverage must be achieved through Unit Tests! You are not allowed to create one E2E test and say it covers everything!

Task Types

The workflow verification strategy changes depending on the **Task Type** you are assigned. We categorize tasks into three types, each requiring a different verification focus and environment setup.
Visit the next section to see the full workflow for each task.

> **Note:** While the core process of validating the *Golden Patch* remains the same, the nature of the tests you write will differ significantly between fixing a bug in an existing repo and building a new tool from scratch.

1. Full Tool Implementation

Test design-agnostic tool from scratch that meets specific functional requirements.

In this task type, you will evaluate whether the tool is being built correctly and whether the test coverage provided in the task is adequate, meaning it exercises all implementation code. Furthermore, you will receive a list of interfaces that you must use as the basis for creating your natural-language, implementation-agnostic prompt.

Examples:

Implement a rate limiter that determines whether a user request is allowed based on a maximum number of requests permitted within a sliding time window. Given a user identifier and a timestamp for a new request, the tool should return whether the request is allowed under the configured limit for that window. Allowed requests must affect future decisions consistently with the configured rate-limiting behavior, while rejected requests must not. The solution should expose a RateLimiter interface in rate_limiter/core.py, initialized with the request limit and window size in seconds, and provide an is_allowed method that takes a user ID and the current timestamp and returns a boolean.

Important Commands for this Project

Below is a list of the commands you **WILL** use in every single task and their use:

- Building the **Docker image:** `docker build --platform linux/amd64 -t <image_name> .`
- Running the **Docker Container on Linux/Mac:**

```shell
docker run --rm -it \
  -v "$(pwd)/audit.sh:/app/audit.sh:ro" \
```

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

```
  -v "$(pwd)/run_script.sh:/app/run_script.sh" \
  -v "$(pwd)/parsing.py:/app/parsing.py" \
  -v "$(pwd)/test_patch.diff:/app/test_patch.diff:ro" \
  -v "$(pwd)/golden_patch.diff:/app/golden_patch.diff:ro" \
  <docker_image_name>
```

- Running the **Docker Container on Windows (PowerShell):**

```shell
docker run --rm -it `
  -v "${PWD}/audit.sh:/app/audit.sh:ro" `
  -v "${PWD}/run_script.sh:/app/run_script.sh" `
  -v "${PWD}/parsing.py:/app/parsing.py" `
  -v "${PWD}/test_patch.diff:/app/test_patch.diff:ro" `
  -v "${PWD}/golden_patch.diff:/app/golden_patch.diff:ro" `
  <docker_image_name>
```

- **Copying the before json file to your local machine:**
  `docker cp <container_id_or_name>:/app/before.json ./before_output.json`
- **Copying the after json file to your local machine:**
  `docker cp <container_id_or_name>:/app/after.json ./after_output.json`

Workflow Overview

For all tasks, you will be provided with a Dockerfile and Docker Image, run_script, parsing.py script, golden_patch.diff, test_patch.diff, before.json, after.json, and interfaces.md. You will then have to download all these files to set up your local environment and make sure the Docker Container and audit.sh script runs correctly.

🚨**Even though we are providing you with all the files, you are still required to check them to make sure they are correct!**🚨

You will also need to create a **prompt** based on the interfaces.md file we are providing. See the specific guidelines for the prompt in the corresponding section.

🚨**If you have to install any missing dependencies to make the code run, make sure to update the Dockerfile as well and upload the new Dockerfile in the task!**🚨

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

1. Full Tool Implementation

In this category, you focus on **one utility tool.** Meaning that you will be working with a brand new codebase and implementing a tool from scratch. Here are the steps for a high quality **Tool Implementation** task.

🚨**If the tool of your task requires cloning the whole repository to work properly, you must skip it and flag it for the team!**🚨

1. Full Tool Implementation Workflow	
01	Download the files provided in the task (Dockerfile and Docker Image, run script, parsing.py script, golden_patch.diff, test_patch.diff, before.json, after.json, and interfaces.md).
02	Verify if the **test_patch.diff** is Implementation-Agnostic. If it's not, you **MUST** modify the tests to make sure they allow other valid solutions and not just the golden patch one.
03	Verify if the **test_patch.diff**'s coverage is adequate. Check this section that describes what a good test coverage should look like. If it's not, you must create new tests (MAKE SURE THEY ARE IMPLEMENTATION-AGNOSTIC). 🚨**Important:** if your tests are Implementation-Specific**,** your task is NOT good and will fail the evals!
04	Verify if the **interfaces.md** is Implementation-Agnostic and if it's documenting all the **minimal external interaction points** present in the golden_patch.diff that are needed for a functional code to be implemented . If it's not, you **MUST** modify the interface list to make sure they are not documenting helper functions or detailing HOW to implement the interfaces requested., and also add the missing interfaces.
05	Craft an **Implementation-Agnostic** prompt by describing WHAT the solution must do, not HOW the model must build it. You still document the relevant external interfaces, such as classes and functions, but only at the level needed to define their role, inputs, outputs, or expected behavior. Please check the Implementation-Agnostic section to understand how to build a good prompt! Make sure your prompt is targeting the interfaces from the interfaces.md file.
06	Paste the interfaces.md content you modified (or not) into the interface field. It's right after the prompt.
07	Build the image: `docker build --platform linux/amd64 -t <image_name> .`
08	Before running the container and mounting, **MAKE SURE** all files (run_script.sh, parsing.py, golden_patch.diff, test_patch.diff, audit.sh) are in the same directory as the one your terminal is running. Run the container using this command.
09	Run the audit script: `bash audit.sh`. 🚨**Important:** If you need to download any other dependencies for the audit.sh script to run. Don't forget to also modify the **Dockerfile** to include those dependencies!
10	Copy the `before.json` and `after.json` from the container to your local machine. Steps:

latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z CONFIDENTIAL

1. Full Tool Implementation Workflow

1. Open a new terminal and keep in mind that the before and after json files will be saved to the current path of your new terminal.
2. Run `docker ps` so you can identify the container name.
3. Run `docker cp <container_id_or_name>:/app/before.json ./before_output.json` to save the before.json file.
4. Run `docker cp <container_id_or_name>:/app/after.json ./after_output.json` to save the after.json file.
5. **Linux/Mac:** run `ls` to check if the before.json and after.json files were correctly copied to your local machine.
   Windows: run `dir` to check if the before.json and after.json files were correctly copied to your local machine.

11 Verify if the **before_output.json** has the same amount of tests as **after_output.json.**
Verify that all tests are failing in **before_output.json.**
Verify that all tests are passing in **after_output.json.**

12 Now you need to compare the **before_output.json** you generate with the **before.json** we provide in the task to make sure they are the same. If they are not, it's probably because you modified the test_patch so you need to update the before.json in the task.

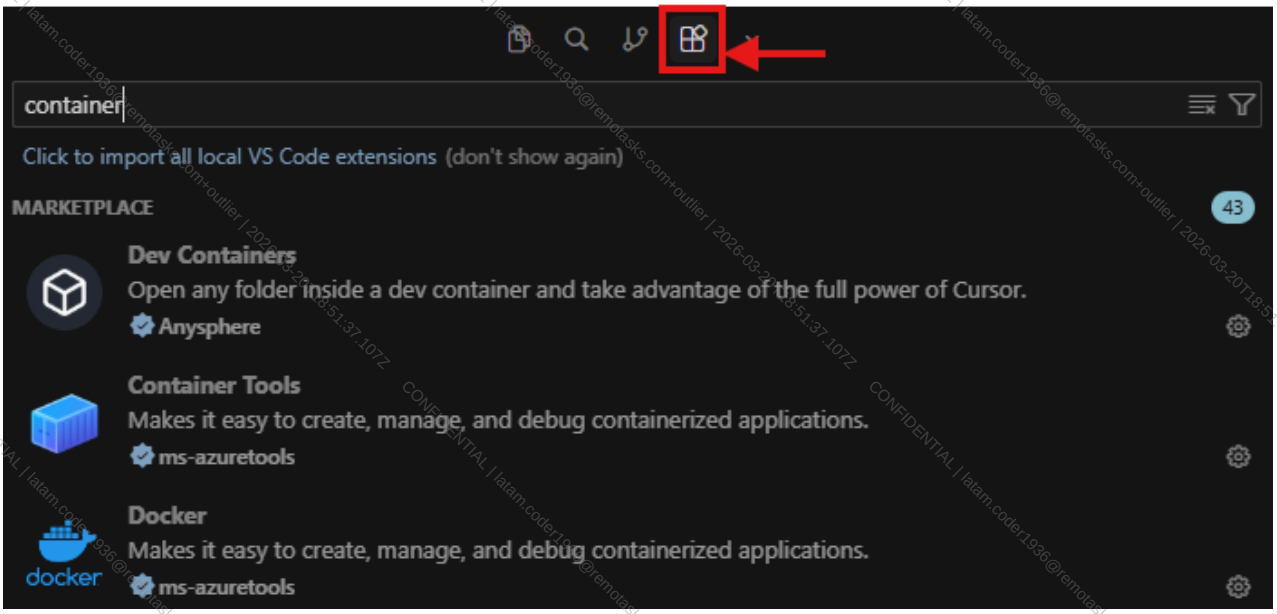
13 You also need to compare the **after_output.json** you generate with the **after.json** we provide in the task to make sure they are the same. If they are not, it's probably because you modified the test_patch so you need to update the after.json in the task.

14 Create Rubrics targeting **YOUR PROMPT**'s requests, not the golden patch.
Make sure you do not use the golden_patch.diff's implementation when writing rubrics or else they will not be Implementation-Agnostic. You are allowed to provide implementation details in rubrics as long as you give them as an example!

🚨**IMPORTANT: MAKE SURE THERE ARE NO TESTS PASSING IN THE BEFORE.JSON!**

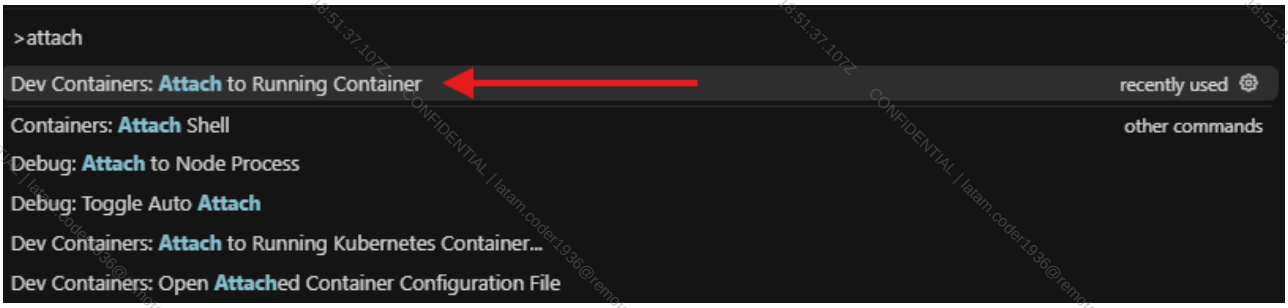
Docker Extensions

If you're working in Cursor/VSCode, **we strongly recommend** installing the following extensions, as they can boost your productivity and improve your workflow:

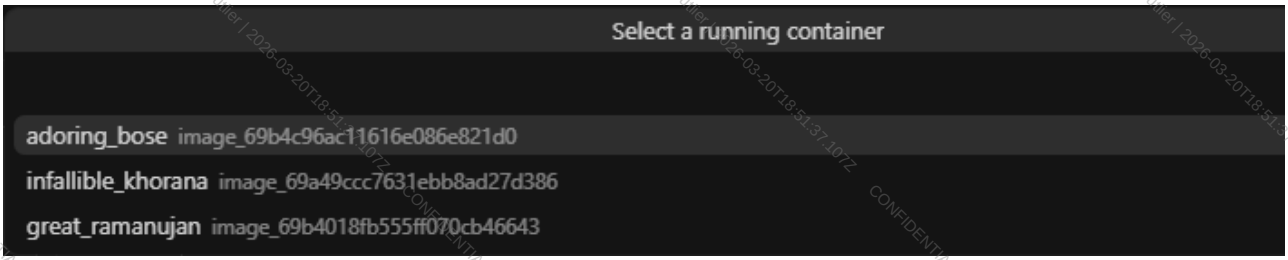


latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z CONFIDENTIAL

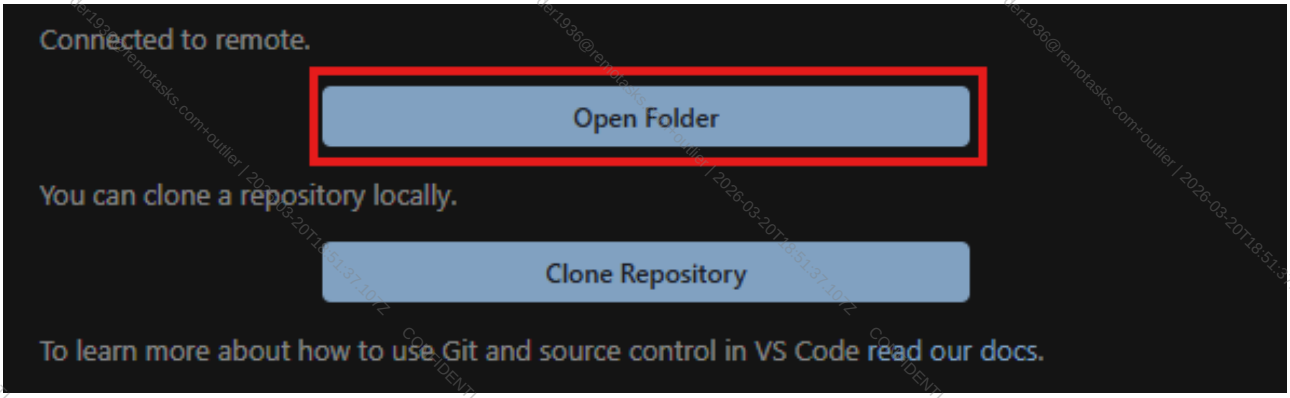
After building your container, press `Ctrl`/`Cmd` + `Shift` + `P`, then select “Attach to Running Container”:



Then select the container in which you want to work:



Wait until the new window loads, press “Open Folder”, and then select the “/app” path:



Rubric Generation

🚨**YOU DON`T NEED TO CREATE RUBRICS FOR THINGS THAT ARE ALREADY BEING COVERED BY UNIT TESTS!**🚨 - *you will not be penalized if you do.*

A rubric is a set of clear, specific requirements that describe what a strong response to your generated prompt must contain. You must create a rubric document tailored specifically to

latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z CONFIDENTIAL

your prompt's requirements with a minimum of 10 criteria, more are encouraged.

You must analyze the task complexity and generate a list of rubrics.

Also, each rubric will have a weight, which represents the relative importance of a specific criterion compared to the others.

> ⚠ **Do NOT select 2 or 4 as weights. The only allowed options are 1, 3, or 5. ⚠**

Rubric Scoring

Rating	Definition
FAIL (Does not meet)	The code fails to run, breaks the build, or hallucinates non-existent libraries. There is at least something that doesn't meet the rubric's intent. Describe what is (are) the failing issue(s).
PASS (Fully meets)	Perfect implementation. Matches the rubric's intent in logic, style, and efficiency. Describe what was observed.

Rubric Weight

Rating	Definition
5 — Mandatory	This is a core requirement for adequately answering the prompt, is clearly expected based on the prompt, and it would be hard to envision an acceptable response without it.
3 — Important	This makes the response substantially better, and is reasonably or **implicitly** expected based on the prompt, though you can envision an acceptable response without it as long as all other criteria are met.
1 — Nice to have (Optional)	This is a nice-to-have and necessary for a perfect response, but a response can still be strong without it.

Rubric Dimension

1. **Instruction Following:** Ensures the response adheres to explicit directions in the prompt (format, constraints, language, libraries, required elements).

latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z CONFIDENTIAL

2. **Code Correctness:** Ensures any code performs the intended task and produces correct results based on the prompt.
3. **Code Quality:** Covers robustness, maintainability, idiomatic patterns, and avoiding fragile or error-prone design choices.
4. **Code Clarity:** Covers readable, well-structured code, including naming, organization, and formatting.
5. **Code Efficiency:** Covers conciseness, avoidance of unnecessary steps, and reduction of redundancy.

Examples

1. Instruction Following

Good examples:

- ✔ "The response uses Pandas for the Data Manipulation of function foo()."
- ✔ "The response outputs the result in the required JSON format (provide the JSON format)."

Bad examples:

- ✘ "The response follows the instructions well." (vague)
- ✘ "The response does not break the rules." (negative)

2. Code Correctness

Good examples:

- ✔ "The response refactors function foo() so it returns x instead of y if the input is z."
- ✔ "The response returns a python dictionary containing x, y, z in the foo function."

Bad examples:

- ✘ "The logic is mostly correct." (vague)
- ✘ "The code does not fail." (negative)

3. Code Quality

Good examples:

- ✔ "The solution adheres to the single responsibility principle by ensuring the X Interface is only responsible for doing Y."

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

✔ “The response uses snake case to name the x function as this is the convention in Python.”

Bad examples:

✘ “The code is high quality.” (non-specific)
✘ “The code is not messy.” (negative)

4. Code Clarity

Good examples:

✔ “The solution creates a docstring for the foo function, mentioning its output.”
✔ “The solution adds `int` as the expected parameter type of the `threads` parameter of the foo function.”

Bad examples:

✘ “The code looks clean.” (subjective)
✘ “The code is not confusing.” (negative)

5. Code Efficiency

Good examples:

✔ “The foo function created is implemented with O(n) complexity.”
✔ “The foo function implemented uses `async` to avoid blocking the event loop during I/O-bound operations.”

Bad examples:

✘ “The code is fast.” (unsupported)
✘ “The solution is not inefficient.” (negative)

Rubric basics

A good rubric is achieved once it is:

Characteristic	Meaning
Verifiable	Can be checked directly from the code.

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

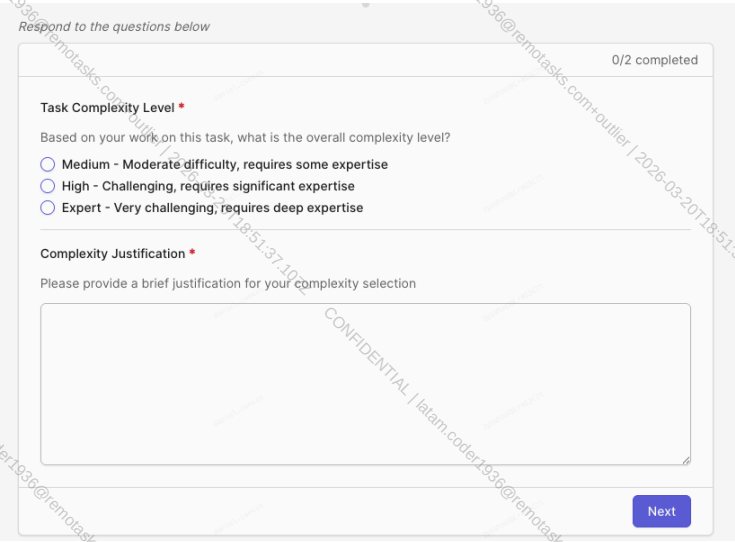
Characteristic	Meaning
Atomic	Tests one single requirement, not multiple.
Self-Contained	Understandable on its own without extra context.
Non-Overlapping	Does not duplicate or repeat another criterion.
Clear & Positive	States what the response should include using simple, positive language (e.g., "Code includes..." vs "Code doesn't forget...").

Complexity Level

There are two **Complexity Level** fields in each task. The first one being the **Complexity Validation** and the second one being the **Complexity Justification.**

In the **Task's Complexity Validation**, you must analyze the golden patch and verify whether the changes are complex enough for you to continue with the task.

In the **Task's Complexity Justification** section, you must categorize the overall difficulty after working on the task. You are expected to have more context of the difficulty level of it and provide a detailed justification on the complexity level of the task.



→ **Medium:** Requires relevant, targeted expertise, not something a junior developer could solve. Moderate difficulty.

→ **High:** Significant full-stack expertise required (e.g., Major version migrations, multi-file architectural changes). Challenging.

→ **Expert:** Deep full-stack expertise required (e.g., Kernel level changes, complex security implementations, proprietary algorithms). Very challenging.

Justification: Explain your choice based on the lines of code changed, the conceptual difficulty, and the dependencies involved.

The Prompt

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

🚨 **IF YOUR PROMPT EVER NEEDS TO COVER GPU USAGE OR CROSS PLATFORM SUPPORT (E.G., WINDOWS AND MACOS), YOU MUST SKIP THE TASK AS THE CONTAINER RUNS ONLY ON LINUX AND IT`S NOT POSSIBLE TO TEST THESE REQUIREMENTS** 🚨

Your **Prompt** must cover the **minimal external interaction points** that are needed to be implemented for a functional solution. These minimal external interfaces are provided in the interfaces.md file but you must still check if there are interfaces that do not belong there (e.g., helper methods, getters, setters, internal implementation details) and remove them, and also check the **golden_patch.diff** to check if there are missing interfaces that are needed for a functional solution to be implemented.

1. The "Architectural" Mindset

When the diff involves complex logic, you must conduct a code review. Do not just ask for "functionality"; ask for **architecture**.

- **Identify the Pattern:** Is the patch implementing a Singleton? A Strategy Pattern? Memoization?
- **Identify the Constraint:** Does the patch introduce thread safety? Does it reduce Time Complexity from O(n^2) to O(n)? Does it handle a specific edge case in asynchronous data flow?
- **Identify the Tooling:** Does the patch introduce a specific library (e.g., NumPy, RxJS, Hibernate) to solve the problem?

2. The Structure of a Complex Prompt

A high-quality prompt is self-contained and comprehensive. It generally follows this flow:

1. **The Objective:** High-level goal (e.g., "Optimize data aggregation").
2. **The Strategy:** The specific technical approach (e.g., "Use a sliding window algorithm").
3. **The Constraints:** High-level implementation details (e.g., "Ensure memory safety," "Use Python's asyncio").
4. **The Expected Interface:** (See Section 3 below).

3. Mandatory Component: Expected Interface

Critically Important: You must include a dedicated section within your rewritten prompt that defines **every** newly introduced file, function, or class that an external module or test suite will interact with. Remember to only document the **minimal external interaction point** needed for a functional solution to be implemented so your prompt remains Implementation-Agnostic.

This section must be structured exactly as follows for each item:

Required Fields

- **Path:** [Exact file path as it appears in the intended structure]

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

- **Name:** [Class.method or function name]
- **Type:** [e.g., class, method, function, or interface]
- **Input:** [Parameters and types, e.g., chunk: GlibcChunk]
- **Output:** [Return type, e.g., None or Promise<void>]
- **Description:** [Brief description of WHAT the interface does]

Language-Specific Fields (Include as applicable)

- **Inheritance (TS/Java):** extends <Base>; implements <IfaceA, IfaceB>
- **Embedding/Implements (Go):** embeds <TypeA>; implements <IfaceA, IfaceB>
- **Bases/Overrides (Python):** bases: <BaseA, BaseB>; overrides: <Base.method>
- **Annotations/Decorators:** @Override, @Inject, @dataclass, @cached_property, @sealed

⚠️ **Failure Criteria for Interfaces**

- **[Fail - Missing Interface Section]:** The section is missing entirely, or omits a required field (Path, Name, Type, etc.) or an applicable language-specific field.
- **[Fail - Undocumented Interface]:** The section is present but incomplete. It fails to document one of the **minimal external interaction points** required for a functional solution to be implemented.

4. Complexity Checklist

Before submitting, verify the prompt against the code changes:

- [ ] Did I mention the specific **design pattern** or **algorithm** used in V2?
- [ ] Did I specify how to handle **errors** or **edge cases** that appear in the patch?
- [ ] Did I specify which **external libraries** or APIs are introduced?

5. Summary Checklist for a High-Quality Prompt

To ensure you are not falling into common traps:

- **Check Domain:** Did I refer to the specific technology stack (e.g., Django, Kubernetes, TensorFlow) correctly?
- **Check Task:** Does my prompt cover ***ALL*** the minimum interfaces required for a functional solution to be implemented while being Implementation-Agnostic?
- **Check Difficulty:** If the patch involves a complex design pattern (like a Singleton or Factory), did I **explicitly ask** for that pattern?
- **Check Interfaces:** Have I listed every public interface with its specific types, inputs, and outputs?

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

Common Error: Confusing "Implementation-Specific" Details with "Observable Public Behavior"

The Scenario:
You are working on a Tool Implementation task to create a Compressed Directed Acyclic Word Graph (DAWG) builder.

The Prompt Requirement:"The Dawg builder must accept ASCII string keys together with integer payloads... Keys outside ASCII must be rejected, insertion must be in strict sorted order, duplicate payload values must be rejected."

The Interface:
The public entry point is the `Dawg.insert(word: str, value: int)` method.

❌ The Wrong Way (The Anti-Pattern)

Action: The contributor removes tests that check for non-ASCII rejection, out-of-order rejection, and duplicate value rejection (e.g., removing `self.assertRaises(ValueError, dawg.insert, "b", 1)` after inserting `"c"`).
Flawed Rationale: "These tests are implementation-specific. They might reject valid alternative DAWG implementations. Tests must assert only on observable public behavior."

Why this fails: This is a fundamental misunderstanding of Black-Box Testing. The contributor confused *error handling on a public interface* with *internal architectural choices*. By removing these tests, explicit prompt requirements (rejecting invalid inputs) are left completely unverified, resulting in a **[Fail - Missing Test]**.

✅ The Right Way (The Best Practice)

Action: The contributor writes or keeps tests that intentionally pass invalid arguments to the public `Dawg.insert()` method and assert that the correct exceptions are raised.

```python
def test_insert_rejects_out_of_order_keys(self):
    dawg = Dawg()
```

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

```
        dawg.insert("cat", 1)
        # Testing observable public behavior: The system must reject this input.
        with self.assertRaises(ValueError):
            dawg.insert("apple", 2)
```

Why this is correct: Testing how a public method responds to invalid inputs (raising an error) is **pure black-box testing**. It does not dictate *how* the code detects the error internally, only that the public interface enforces the contract requested by the prompt.

🧠 Core Concept: Where is the Boundary?

To adhere to the **Generalizable Verification** constraints, you must understand the difference between testing the *interface* versus testing the *internals*:

- **Implementation-Specific (Do NOT Test in Code):**
  - Checking if the `Dawg` internally uses a specific `dict` or `list` to store nodes.
  - Asserting that a private helper function like `_minimize()` was called 3 times.
  - Checking the exact byte-size of the internal state before `finish()` is called.
  - *Note: If the prompt explicitly mandates an internal architectural choice (e.g., "must use a sliding window algorithm"), verify it using **Rubrics**, not test scripts.*
- **Observable Public Behavior (MUST Test in Code):**
  - Passing an out-of-order string to the public `Dawg.insert()` method and expecting a `ValueError`.
  - Verifying that `Dawg.finish()` returns the exact byte format requested.
  - Checking that `lookup()` successfully finds a previously inserted key.

The Golden Rule: If the prompt explicitly defines constraints on inputs (e.g., "must reject X"), your test suite *must* feed the public interface "X" and assert that it fails gracefully. Error handling is an observable system state!

Audit Script

Below is the audit.sh script that you must run to ensure that your task was properly set up

latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z CONFIDENTIAL

and everything runs as expected. You must copy and paste this script's output into your task (there is a specific field for it). Here is the link for you to download the audit script: [script](#).

```shell
#!/usr/bin/env bash
set -euo pipefail

cd /app

# Check that the Dockerfile does not create non-dependency files
echo "Checking Dockerfile for non-dependency file creation..."

if [[ ! -f "Dockerfile" ]]; then
  echo "WARNING: No Dockerfile found at /app/Dockerfile; skipping non-dependency
file check." >&2
else
  python3 << 'PYEOF'
import re, sys

ALLOWED_BASENAMES = {
    # Rust
    "cargo.toml", "cargo.lock",
    # Node / JS
    "package.json", "package-lock.json", "yarn.lock", "pnpm-lock.yaml",
    ".npmrc", ".yarnrc", ".yarnrc.yml",
    # Python
    "requirements.txt", "pipfile", "pipfile.lock", "pyproject.toml",
    "setup.py", "setup.cfg", "poetry.lock", "constraints.txt",
    # Ruby
    "gemfile", "gemfile.lock",
    # Go
    "go.mod", "go.sum",
    # Java / JVM
    "pom.xml", "build.gradle", "build.gradle.kts", "settings.gradle",
    "settings.gradle.kts", "gradle.properties", "gradlew", "gradlew.bat",
    # .NET / C#
    "nuget.config", "packages.config", "directory.build.props",
    "directory.packages.props",
    # PHP
    "composer.json", "composer.lock",
```

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

```
    # Swift / CocoaPods
    "podfile", "podfile.lock", "package.swift", "package.resolved",
    # Elixir / Erlang
    "mix.exs", "mix.lock", "rebar.config", "rebar.lock",
    # Clojure
    "project.clj", "deps.edn", "shadow-cljs.edn",
    # R
    "description", "renv.lock",
    # Dart / Flutter
    "pubspec.yaml", "pubspec.lock",
}

ALLOWED_EXTENSIONS = {
    ".gemspec", ".cabal", ".csproj", ".fsproj", ".vbproj", ".sln", ".toml",
}

def is_dependency_file(path: str) -> bool:
    basename = path.rstrip("/").split("/")[-1].lower()
    if basename in ALLOWED_BASENAMES:
        return True
    _, _, ext = basename.rpartition(".")
    if ext and f".{ext}" in ALLOWED_EXTENSIONS:
        return True
    if re.match(r"requirements.*\.txt$", basename):
        return True
    return False

COPY_ADD_RE = re.compile(
    r"^\s*(?:COPY|ADD)\s+(?:--\S+\s+)*(.+?)\s+(\S+)\s*$",
    re.IGNORECASE,
)
REDIRECT_RE = re.compile(r"(?:>>?|tee\s+)\s*([^\s;|&><]+)")
TOUCH_RE = re.compile(r"\btouch\s+((?:[^\s;|&><]+\s*)+)")

violations = []

with open("Dockerfile") as fh:
    lines = fh.read().splitlines()
```

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

```
i = 0
while i < len(lines):
    logical = lines[i].rstrip()
    while logical.endswith("\\"):
        i += 1
        if i >= len(lines):
            break
        logical = logical[:-1] + " " + lines[i].rstrip()

    m = COPY_ADD_RE.match(logical)
    if m:
        tokens = [t for t in logical.split()[1:] if not t.startswith("--")]
        if len(tokens) >= 2:
            for src in tokens[:-1]:
                src_base = src.rstrip("/").split("/")[-1]
                if src_base in (".", "..") or src.endswith("/"):
                    violations.append(f"Line {i+1}: COPY/ADD copies entire
directory '{src}' (may include non-dependency files)")
                elif not is_dependency_file(src_base):
                    violations.append(f"Line {i+1}: COPY/ADD copies non-dependency
file '{src}'")
    elif re.match(r"^\s*RUN\b", logical, re.IGNORECASE):
        shell_part = re.sub(r"^\s*RUN\s+", "", logical, flags=re.IGNORECASE)

        # Check shell redirections (>, >>, tee)
        for m in REDIRECT_RE.finditer(shell_part):
            dest = m.group(1).strip("'\"")
            dest_base = dest.rstrip("/").split("/")[-1]
            if dest_base and not is_dependency_file(dest_base):
                violations.append(f"Line {i+1}: RUN writes non-dependency file
'{dest}' via shell redirection")

        # Check touch commands
        for m in TOUCH_RE.finditer(shell_part):
            for touched_file in m.group(1).split():
                dest_base = touched_file.rstrip("/").split("/")[-1]
                if dest_base and not is_dependency_file(dest_base):
                    violations.append(f"Line {i+1}: RUN creates non-dependency
file '{touched_file}' via touch")
```

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

```
    i += 1

if violations:
    print(f"WARNING: Dockerfile references {len(violations)} file(s) that may not
be dependency files — please verify manually:", file=sys.stderr)
    for v in violations:
        print(f"  - {v}", file=sys.stderr)
else:
    print("Dockerfile check passed: no non-dependency file creation detected.")
PYEOF
fi

# Validate required files exist
missing=()
for f in test_patch.diff golden_patch.diff run_script.sh parsing.py; do
  [[ -f "$f" ]] || missing+=("$f")
done
if (( ${#missing[@]} )); then
  echo "ERROR: Missing required file(s): ${missing[*]}" >&2
  exit 1
fi

echo "Applying test patch..."

git apply --ignore-space-change --ignore-whitespace test_patch.diff
echo "Running run_script.sh..."
bash run_script.sh 1> stdout.txt 2> stderr.txt || true
echo "Parsing before.json..."
python3 parsing.py stdout.txt stderr.txt before.json

echo "Applying golden patch..."
git apply --ignore-space-change --ignore-whitespace golden_patch.diff
echo "Running run_script.sh..."
bash run_script.sh 1> stdout.txt 2> stderr.txt || true
echo "Parsing after.json..."
python3 parsing.py stdout.txt stderr.txt after.json

echo "Computing fail-to-pass and pass-to-pass tests..."
python3 << 'PYEOF'
```

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

```python
import json, sys

FAIL_STATUSES = {'FAILED', 'ERROR', 'SKIPPED'}

def normalize(status):
    return 'FAILED' if status in FAIL_STATUSES else status

with open('before.json') as f:
    before = {t['name']: normalize(t['status']) for t in json.load(f)['tests']}
with open('after.json') as f:
    after = {t['name']: normalize(t['status']) for t in json.load(f)['tests']}

f2p = [name for name, status in before.items()
       if status == 'FAILED' and after.get(name) == 'PASSED']

new_passes = [name for name, status in after.items()
              if name not in before and status == 'PASSED']
f2p.extend(new_passes)

p2f = [name for name, status in before.items()
       if status == 'PASSED' and after.get(name) == 'FAILED']

p2p = [name for name, status in before.items()
       if status == 'PASSED' and after.get(name) == 'PASSED']

# Check for any FAILED tests in after.json
failed_in_after = [name for name, status in after.items() if status == 'FAILED']

with open('/app/fail_to_pass.json', 'w') as f:
    json.dump(f2p, f, indent=2)
with open('/app/pass_to_pass.json', 'w') as f:
    json.dump(p2p, f, indent=2)

print(json.dumps({'fail_to_pass': f2p}, indent=2))
if new_passes:
    print('{} new test(s) appeared and PASSED (counted as
fail-to-pass).'.format(len(new_passes)))
print('{} total test(s) went from FAILED to PASSED.'.format(len(f2p)))
print('{} test(s) stayed PASSED (pass-to-pass).'.format(len(p2p)))
```

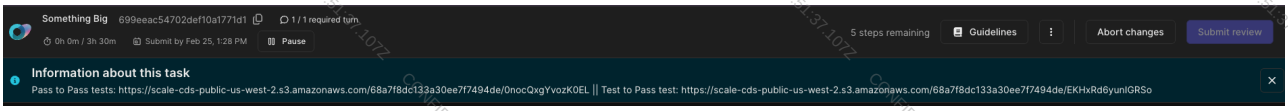
CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

```
if failed_in_after:
    print('ERROR: {} test(s) have FAILED status in
after.json:'.format(len(failed_in_after)), file=sys.stderr)
    for t in failed_in_after:
        print('  - {}'.format(t), file=sys.stderr)
    sys.exit(1)

if p2f:
    print('ERROR: {} test(s) went from PASSED to FAILED
(regressions):'.format(len(p2f)), file=sys.stderr)
    for t in p2f:
        print('  - {}'.format(t), file=sys.stderr)
    sys.exit(1)

if not f2p:
    print('ERROR: No fail-to-pass tests found.', file=sys.stderr)
    sys.exit(1)
PYEOF

echo "verification_passed"
echo "Done."
```

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z

Extra Validation Step [FOR REVIEWERS ONLY]

At the top of the task, there will be a dialog box with two links as shown below:



You are required to download these two files, rename them to **pass_to_pass.json** and **fail_to_pass.json** so you can open them (if you do not add `.json` they will not open).

Then, you are required to compare the **P2P** and **F2P** tests in those files with the **before.json** and **after.json** files that the CB provided to see if they match.

After verifying it, you will have a final step in the taxonomy to provide the **pass_to_pass.json** and **fail_to_pass.json** files and mark if they are valid or not (not valid means that they do not match the **P2P** and **F2P** tests provided by the CB, so you have to fix the task).

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-20T18:51:37.107Z